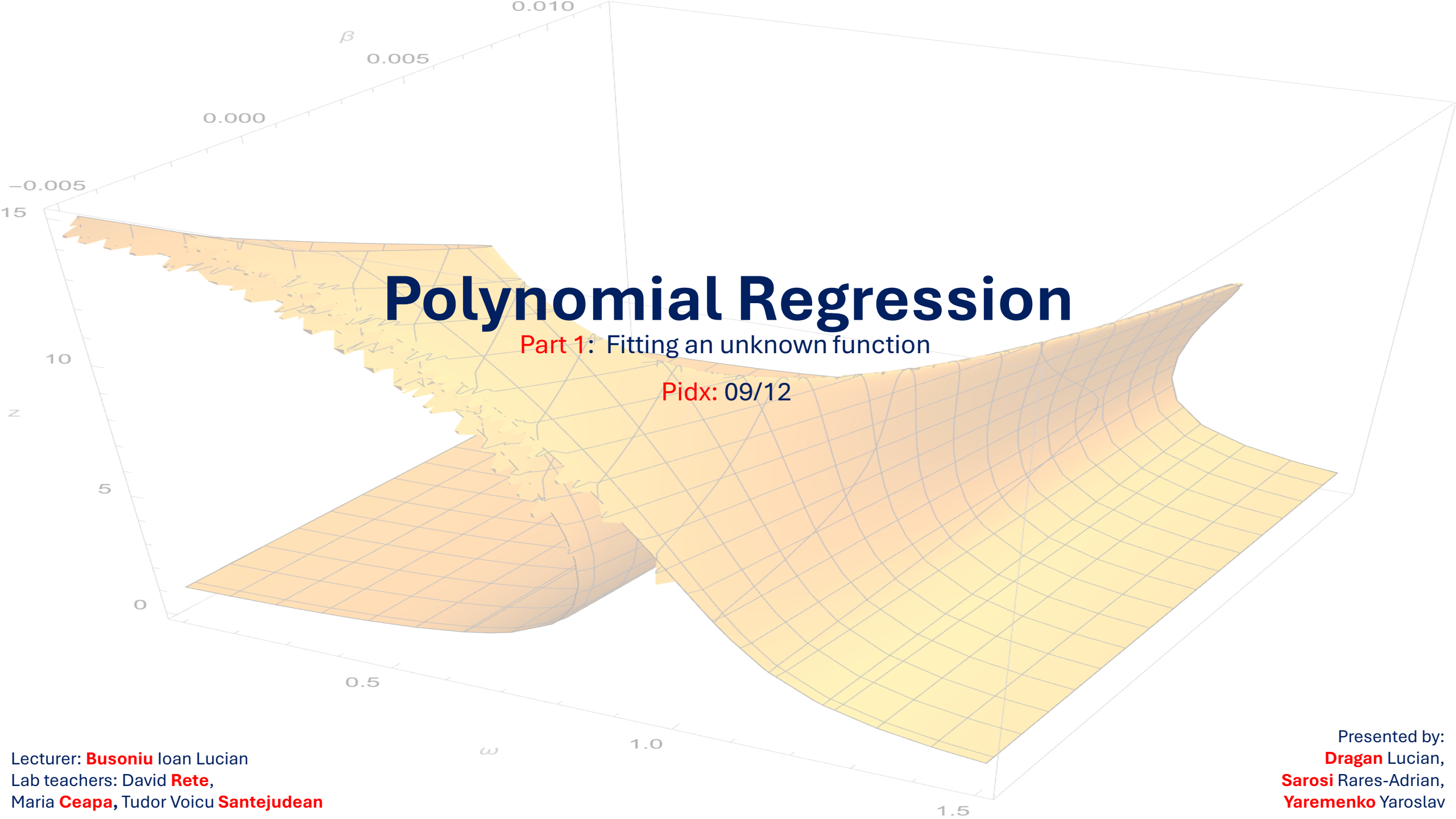


Polynomial Regression

Part 1: Fitting an unknown function

Pidx: 09/12

Lecturer: **Busoniu** Ioan Lucian
Lab teachers: David **Rete**,
Maria **Ceapa**, Tudor Voicu **Santejudean**

Presented by:
Dragan Lucian,
Sarosi Rares-Adrian,
Yaremenko Yaroslav

Presentation Structure

- Introduction & Motivation
- Methodology & Algorithm
- Analysis & Results
- Discussion & Conclusion

Introduction & Motivation

Goal: Build a polynomial approximator that is trained to get close to an unknown function of two inputs.
Find the degree of the polynomial to get the most accurate approximation while avoiding overfitting

$$
\begin{aligned}
g(x_{ij}) = {} & \theta_0 (x_{1i}^0 \cdot x_{2j}^0) + \theta_1 (x_{1i}^0 \cdot x_{2j}^1) + \theta_2 (x_{1i}^0 \cdot x_{2j}^2) + \dots + \theta_m (x_{1i}^0 \cdot x_{2j}^m) + \\
& + \theta_{m+1} (x_{1i}^1 \cdot x_{2j}^0) + \theta_{m+2} (x_{1i}^1 \cdot x_{2j}^1) + \theta_{m+3} (x_{1i}^1 \cdot x_{2j}^2) + \dots + \theta_{2m} (x_{1i}^1 \cdot x_{2j}^{m-1}) + \\
& + \theta_{2m+1} (x_{1i}^2 \cdot x_{2j}^0) + \theta_{2m+2} (x_{1i}^2 \cdot x_{2j}^1) + \theta_{2m+3} (x_{1i}^2 \cdot x_{2j}^2) + \dots + \theta_{3m-1} (x_{1i}^2 \cdot x_{2j}^{m-2}) + \\
& \cdots \\
& + \theta_N (x_{1i}^m \cdot x_{2j}^{m-m})
\end{aligned}
$$

$$
g(x_{ij}) = \underbrace{\left[x_{1i}^0 \cdot x_{2j}^0 \;\middle|\; x_{1i}^0 \cdot x_{2j}^1 \;\middle|\; \dots \;\middle|\; x_{1i}^1 \cdot x_{2j}^0 \;\middle|\; x_{1i}^1 \cdot x_{2j}^1 \;\middle|\; \dots \;\middle|\; x_{1i}^m \cdot x_{2j}^{m-m} \right]}_{\varphi^T(x_{ij})} \cdot \underbrace{\begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_N \end{bmatrix}}_{\theta}
$$

Introduction & Motivation

The Φ matrix (**regressor matrix**) of **size N1*N2, N** (lengths of x1, x2 and the number of parameters)

$$\Phi = \begin{array}{c} \\ \\ \\ \end{array}\begin{array}{c} \overset{N}{\begin{bmatrix} \cdot & \cdot & \cdot \\ & \varphi^T(x_{ij}) & \\ \cdot & \cdot & \cdot \end{bmatrix}} \end{array}\begin{array}{l} \\ N1 \\ X \\ N2 \end{array}$$

N - number of parameters

m	2	3	4	5
N	6	10	15	21

+4 +5 +6

$$\Rightarrow \boxed{N = \frac{(m+1)(m+2)}{2}}$$

ex: $m = 4 \longrightarrow N = \underset{1+2+3}{6} + 4 + 5 = 1+2+3+4+\underset{m+1}{5}$ Gauss Sum

Methodology & Algorithm

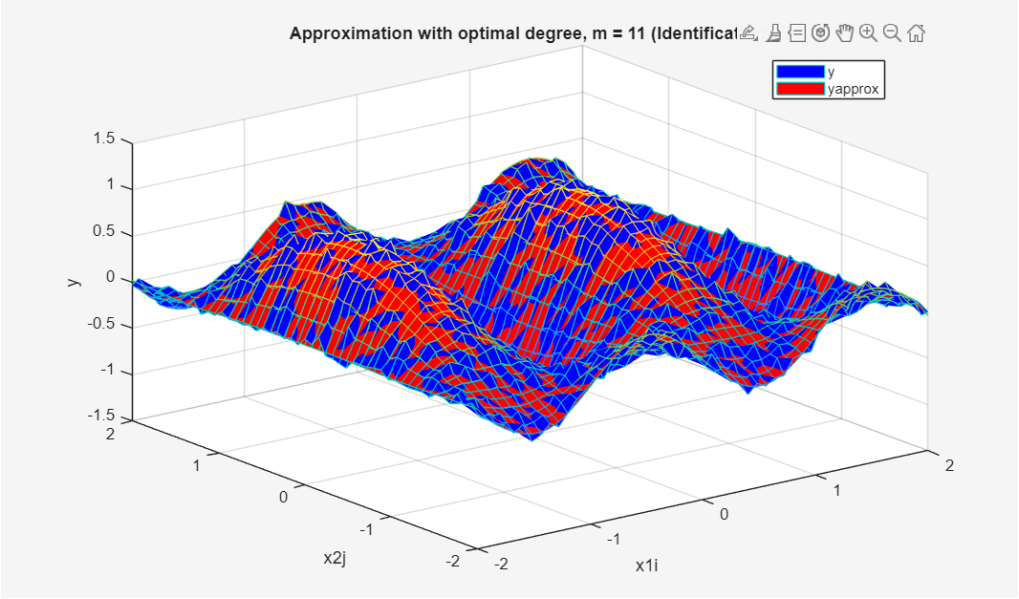
phiCalc → reshape → computeReg → For loop → mesh

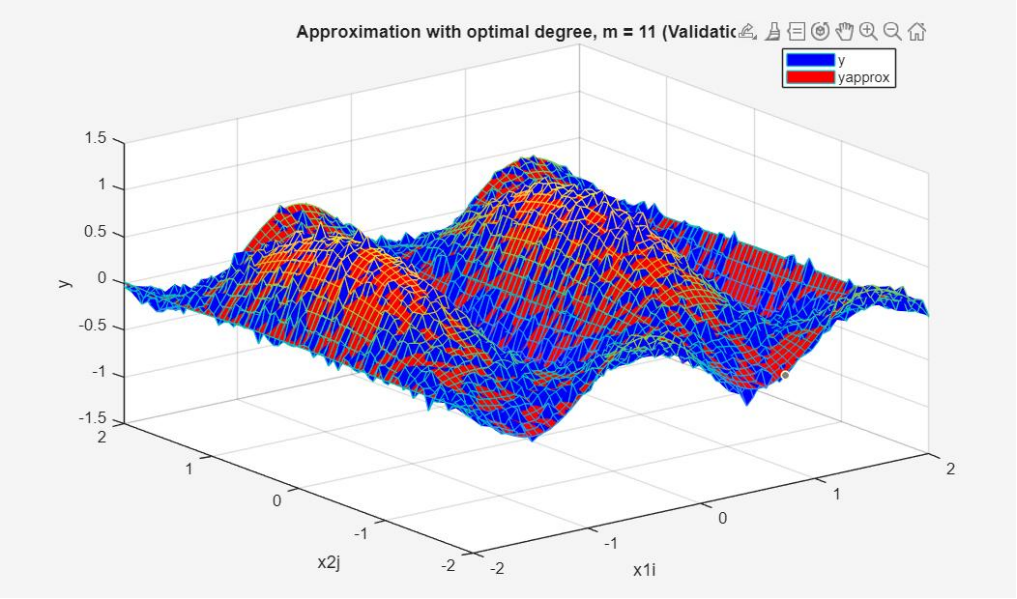
Loop degrees m = 1…n

- **phiCalc:** calculate the number of parameters and the ϕ regressor matrix
- **reshape:** makes a column matrix from the square matrix of output data
- **computeReg:** computes the actual approximated output of the linear regression
- **For loop:** inside this big for loop every degree m is evaluated and the MSE stored in a vector
- **mesh:** used to show the approximated and real outputs in 3D plots

Results: Surface Approximation

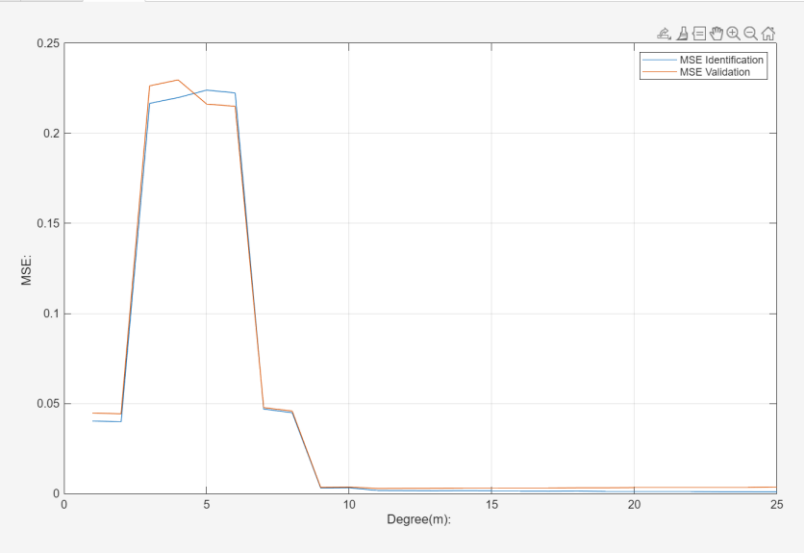
Side-by-side comparison of **Identification** and **Validation** real and approximated data:





Results: Validation Error

- The lowest Mean Square Error (MSE) encountered for the validation dataset is 0.0029.
- This value gives us the optimal degree m=11



MSEMat

25×2 double

	1	2
1	0.0403	0.0448
2	0.0399	0.0443
3	0.2166	0.2263
4	0.2198	0.2296
5	0.2240	0.2162
6	0.2224	0.2150
7	0.0470	0.0478
8	0.0449	0.0459
9	0.0031	0.0035
10	0.0033	0.0037
11	0.0017	0.0029
12	0.0017	0.0030
13	0.0016	0.0030
14	0.0017	0.0031
15	0.0016	0.0032
16	0.0014	0.0032
17	0.0014	0.0031
18	0.0015	0.0032
19	0.0012	0.0033
20	0.0012	0.0034

Observation:

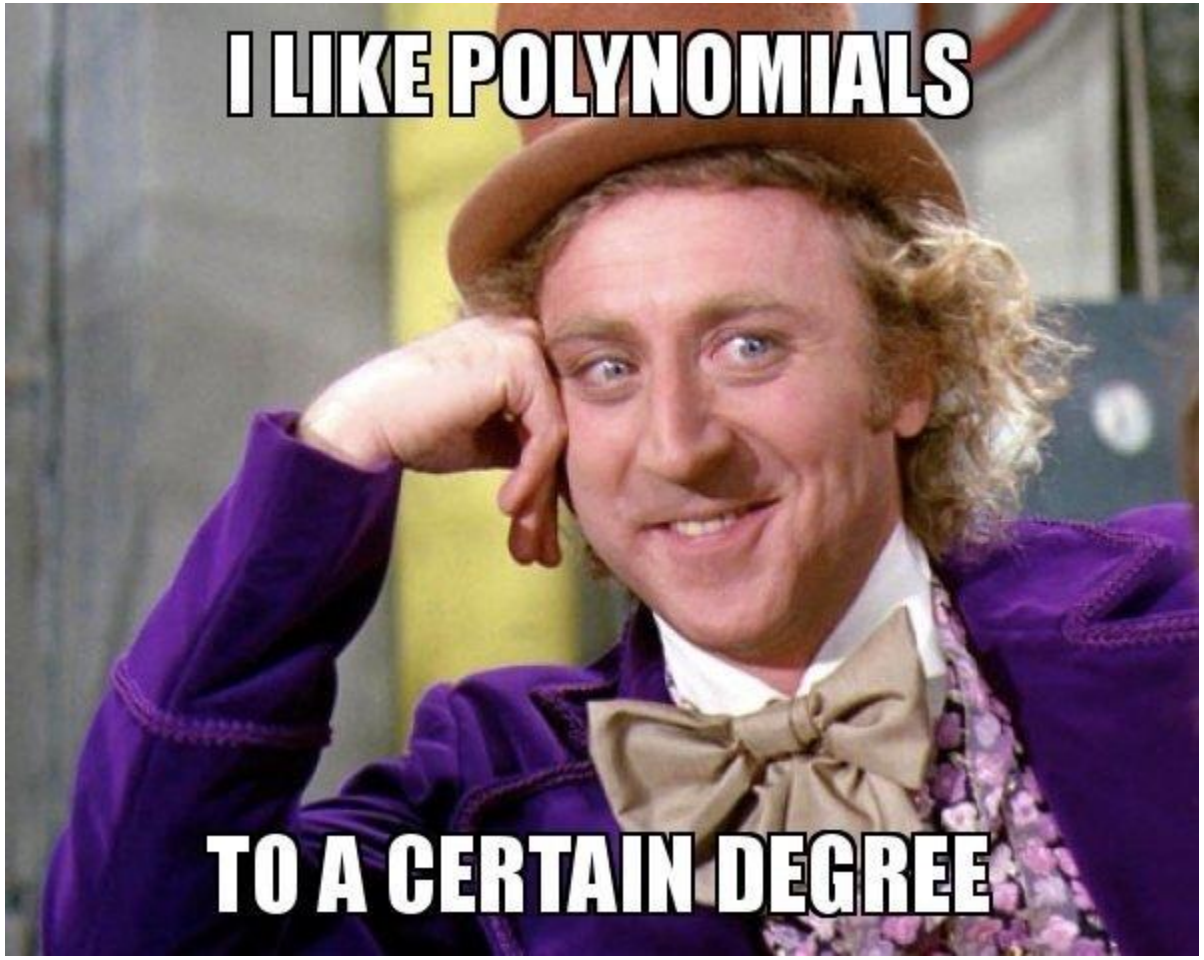
- Even though the MSE values for the identification dataset drop as low as 0.0012 for m = 20, they keep increasing for the validation data due to overfitting.

Discussion & Conclusion:

Small degree → model is too simple, misses important details
Very big degree → model becomes too “wiggly”, follows the noise
We test several degrees and pick the one that works best on validation data

- Built a polynomial model for an unknown 3D surface.
- Use user-defined function to make the code more readable and avoid repeating the same sequence of code
- Used Gauss method (normal equations) to find the coefficients
- Tested several degrees and picked the best using validation MSE
- The model follows the true surface well, without chasing the noise
- Useful in system identification, control, and general regression tasks. Mostly used in AI.

Thank you for your attention!



Code figures:

```
function [PHI] = phicalc(m,x1,x2)
    N1 = length(x1);
    N2 = length(x2);
    N = (m+1)*(m+2)/2;
    k = 0;
    PHI = zeros(N1*N2, N);
    for i = 1:N1
        for j = 1:N2
            elem = 0;
            k = k + 1;
            for pow1 = 0:m
                for pow2 = 0:(m-pow1)
                    elem = elem + 1;
                    PHI(k,elem) = x1(i)^(pow1) * x2(j)^(pow2);
                end
            end
        end
    end
end
```

Listing 1 - “phicalc” function

```
function [Y] = reshapeY(y)
    N1 = length(y);
    N2 = length(y);
    Y = zeros(N1*N2,1);
    k = 0;
    for i = 1:N1
        for j = 1:N2
            k = k + 1;
            Y(k,1) = y(i,j);
        end
    end
end
```

Listing 2 - “reshapeY” function

```
function [Yhat] = computeReg(PHI, y, theta)
    N1 = length(y);
    N2 = length(y);
    yhat = PHI * theta;
    Yhat = zeros(N1, N2);
    k = 0;
    for i = 1:N1
        for j = 1:N2
            k = k+1;
            Yhat(i, j) = yhat(k);
        end
    end
end
```

Listing 3 - “computeReg” function

```
x1i_id = id.X{1};
x2j_id = id.X{2};
y_id = id.Y;

x1i_val = val.X{1};
x2j_val = val.X{2};
y_val = val.Y;
```

Listing 4 – Preparing data

```
for m = 1:25
    PHI_id = phicalc(m, x1i_id,x2j_id);
    Y_id = reshapeY(y_id);
    theta = PHI_id \ Y_id;
    Yhat_id = computeReg(PHI_id, y_id, theta);

    PHI_val = phicalc(m, x1i_val, x2j_val);
    Yhat_val = computeReg(PHI_val, y_val, theta);

    sum1 = 0;
    sum2 = 0;

    for i = 1:length(Yhat_val)
        sum1 = sum1 + norm(Yhat_val(i)-y_val(i))^2;
    end

    for i = 1:length(Yhat_id)
        sum2 = sum2 + norm(Yhat_id(i)-y_id(i))^2;
    end

    MSEvalvect(m) = sum1 / length(Yhat_val);
    MSEidvect(m) = sum2 / length(Yhat_id);
end

[minMSE, mOpt] = min(MSEvalvect);
disp(['Minimum MSE: ', num2str(minMSE), ' at polynomial degree: ', num2str(mOpt)]);
```

Listing 5 – Searching and displaying minimum MSE and optimal degree

```
% Displaying the results
figure, mesh(x1i_id, x2j_id, y_id, 'FaceColor', 'b');
hold on, mesh(x1i_id, x2j_id, Yhat_id, 'FaceColor', 'r');
xlabel("x1i")
ylabel("x2j")
zlabel("y")
legend('y', 'yapprox')
title("Approximation with optimal degree, m = 11 (Identification)")

figure, mesh(x1i_val, x2j_val, y_val, 'FaceColor', 'b');
hold on, mesh(x1i_val, x2j_val, Yhat_val, 'FaceColor', 'r');
xlabel("x1i")
ylabel("x2j")
zlabel("y")
legend('y', 'yapprox')
title("Approximation with optimal degree, m = 11 (Validation)")

% Displaying MSE
figure,
plot(MSEvalvect, 'Color', 'r')
hold on
plot(MSEidvect, 'Color', 'b')
legend('Validation MSE', 'Identification MSE')
```

Listing 6 – Plotting the approximations vs real data and MSE variation